KASPARRO AI

# Backend Engineering

## Selection Assignment

Build NexusAPI — A Multi-Tenant Credit-Gated Backend

| TIME ALLOWED | SUBMISSION FORMAT | QUESTIONS |
|---|---|---|
| **3–5 Days** | **GitHub + Live URL + DECISIONS.md** | **rajat@kasparro.com** |

You are applying for a backend engineering internship at Kasparro AI. This assignment tests whether you can build production-shaped backend systems: systems that handle failure, enforce rules consistently, and are designed to be understood by someone other than you. You will have 5–7 days. Read every section before writing a single line of code.

**“Build something you can explain.”**

*You are allowed to use AI tools. You will be asked to explain every decision you made.*

## The Context

You are applying for a backend engineering internship. The company builds an AI platform used by businesses to measure and improve how they appear across AI-powered search engines. The backend infrastructure — authentication, data pipelines, job processing, API design — is what keeps the platform running.

This assignment tests whether you can build production-shaped backend systems: systems that handle failure, enforce rules consistently, and are designed to be understood by someone other than you.

You will build a focused slice of NexusAPI: a multi-tenant, credit-gated backend API. The word ‘multi-tenant’ means multiple organisations use the same system, and their data must never mix. The word ‘credit-gated’ means API access costs credits, and the system must enforce that correctly — including in situations where things go wrong.

| On using AI tools | *You are allowed and encouraged to use AI coding assistants (Gemini, Copilot, or others) throughout this assignment.*<br>*The written decision log you submit will be used to assess whether you understood what you built. A student who used AI well will be able to explain every decision clearly. A student who accepted AI output without understanding it will not.*<br>*There is no penalty for using AI. There is a significant penalty for submitting work you cannot explain.* |
|---|---|

## What You Are Building

NexusAPI is a backend platform with the following shape. Read this section completely before writing any code.

| Component | What it does |
|---|---|
| **Organisations** | The top-level tenant. Every user belongs to one organisation. Data from one organisation must never be visible to another. This is enforced at the database query level, not just in application code. |
| **Users** | Belong to an organisation. Authenticate via Google OAuth. Receive a signed JWT token on login. All protected endpoints validate this token on every request. |
| **Credit ledger** | Each organisation has a credit balance. Credits are tracked via a transaction log — every addition and every deduction is recorded as a row, never as a direct balance update. The current balance is derived from the ledger. |
| **Product endpoints** | Two endpoints that represent AI-powered features. Each costs a fixed number of credits per call. Access is blocked if the organisation does not have sufficient credits. The deduction must be atomic. |
| **Background job** | One of the two product endpoints must process its work asynchronously. It returns a job ID immediately. The caller polls a separate endpoint for the result. |

## Requirements

The requirements are organised into three tiers. There is no label on these tiers in your submission — but you should know they exist. Most candidates will complete Tier 1. Fewer will complete Tier 2 fully. Tier 3 is intentionally hard. Partial completion of a higher tier is better than not attempting it.

### Tier 1 — Core Backend

*Authentication · Database · Credit system · Protected endpoints*

**R1**
**Project setup**

- Python 3.11+ with FastAPI and async SQLAlchemy
- PostgreSQL database
- All configuration (database URL, secret keys, OAuth credentials) loaded from environment variables. No hardcoded values anywhere in the codebase.
- A GET /health endpoint that returns the application status and confirms the database is reachable
  - *If the database is unreachable, /health must return HTTP 503, not 200*

**R2**
**Database schema**

- Organisations table: id (UUID), name, slug, created_at
- Users table: id (UUID), email, name, google_id, organisation_id (foreign key), role (admin or member), created_at
- CreditTransactions table: id (UUID), organisation_id (foreign key), user_id (foreign key), amount (integer, positive or negative), reason (text), idempotency_key (unique, nullable), created_at
- Every table must have appropriate indexes
  - *The current credit balance of an organisation is computed from the sum of CreditTransactions.amount for that organisation. There is no separate balance column. Explain why in your decision log.*

⚠ **A schema change after you have data is a versioning event, not an edit. Use Alembic for migrations from the start.**

**R3**

- GET /auth/google — redirects the user to Google login

**Google OAuth and JWT**

- GET /auth/callback — receives the Google token, extracts email and name
- If no organisation exists for the user's email domain, create one and make the user an admin
- If an organisation already exists for that domain, add the user as a member
- Return a signed JWT containing: user_id, organisation_id, role, expiry (24 hours)
  - *If Google auth fails or returns an invalid token, return HTTP 401 with a clear error message. Do not return a 500.*

**R4**
**JWT middleware**

- A reusable dependency function that validates the JWT on every protected endpoint
- Returns the current user object if the token is valid
- Returns HTTP 401 if the token is missing, expired, or tampered with
- Returns HTTP 403 if the token is valid but the user's role does not permit the action
  - *Test this explicitly: a valid token from Organisation A must not be usable to access Organisation B's data under any circumstances.*

**R5**
**Credit system**

- POST /credits/grant (admin only) — adds credits to the organisation. Requires: amount (integer), reason (string)
- GET /credits/balance — returns the organisation's current balance and last 10 transactions
- A service function deduct_credits(org_id, amount, reason, idempotency_key) that: checks the balance is sufficient, deducts atomically, logs the transaction, raises InsufficientCreditsError if balance is too low
- deduct_credits must run inside a database transaction. Partial deductions are not allowed.
  - *What happens if deduct_credits is called twice simultaneously for the same organisation with exactly enough credits for one call? Handle this explicitly and explain your approach in the decision log.*

## Tier 2 — Product Endpoints and Failure Handling

*Credit gates · Background jobs · Edge cases*

**R6**
**Product endpoint — synchronous**

- POST /api/analyse — costs 25 credits per call
- Accepts: { "text": "string (10–2000 characters)" }
- Deducts 25 credits before doing any work. If deduction fails, returns HTTP 402 with: { "error": "insufficient_credits", "balance": <current>, "required": 25 }
- On success, returns: { "result": "Analysis complete. Word count: X. Unique words: Y.", "credits_remaining": Z }
  - *What happens if the deduction succeeds but the processing fails? Credits were spent but no result was returned. Handle this explicitly and explain your decision.*

**R7**
**Product endpoint — async**

- POST /api/summarise — costs 10 credits per call
- Returns a job_id immediately (under 200ms). Does not wait for processing to complete.
- Accepts: { "text": "string (10–2000 characters)" }
- Job processing runs in the background using ARQ and Redis
- GET /api/jobs/{job_id} — returns: status (pending / running / completed / failed), result if completed, error if failed
  - *Only the organisation that created the job may retrieve its result. A job_id from Organisation A must not be retrievable by Organisation B.*

⚠ **If the background worker fails and the job never completes, what happens to the 10 credits that were deducted? Handle this explicitly.**

**R8**
**Rate limiting**

- Maximum 60 requests per minute per organisation across all product endpoints combined
- When the limit is exceeded: return HTTP 429 with a Retry-After header (seconds until the window resets)
- The rate limit is per organisation — not per user. If one organisation has 5 users all calling simultaneously, they share one bucket.
  - *What happens if Redis is unavailable? Does the entire API go down, or does it fail open (allow all requests)? Choose one, implement it, and justify your decision in the decision log.*

**R9**
**Structured logging and error**

- Every request logs: timestamp, method, path, organisation_id, user_id, response status, duration_ms
- All log entries are structured JSON — not plain text strings
- No endpoint returns a raw Python stack trace to the caller under any circumstances

**handling**

- All error responses follow a consistent shape: { "error": "<code>", "message": "<human-readable>", "request_id": "<uuid>" }
  - *Include request_id in every error response. This allows the caller to reference a specific request when reporting a problem.*

## Tier 3 — Idempotency

*The hard requirement*

**Read this carefully**

*This requirement is intentionally harder than everything above. Most candidates will not implement it fully. A partial implementation with a clear written explanation of what you attempted and where you got stuck is more valuable than no attempt.*

*Do not skip it without trying.*

**R10**
**Idempotent credit deduction**

- The POST /api/analyse and POST /api/summarise endpoints must accept an optional Idempotency-Key header
- If a request is submitted with an Idempotency-Key that was already used by the same organisation in the last 24 hours: return the original response. Do not deduct credits again.
- If a request is submitted without an Idempotency-Key: process normally. No idempotency guarantee.
- The idempotency check and the credit deduction must happen atomically. It must be impossible for a duplicate request to deduct credits even if two identical requests arrive simultaneously.
  - *The idempotency_key column in CreditTransactions has a unique constraint. Explain in your decision log how you are using this constraint to enforce idempotency at the database level, not just in application code.*

⚠ **This is the kind of problem that exists in every real payment and credit system. Getting it right requires thinking about what happens when two requests arrive at exactly the same time.**

# API Reference

Your implementation must expose the following endpoints with these exact paths and methods. Additional endpoints are permitted but not required.

| Method | Path | Auth | Description |
|---|---|---|---|
| **GET** | `/health` | None | Health check. Returns 200 if healthy, 503 if database unreachable. |
| **GET** | `/auth/google` | None | Initiates Google OAuth flow. |
| **GET** | `/auth/callback` | None | Handles Google OAuth callback. Returns JWT on success. |
| **GET** | `/me` | JWT | Returns the authenticated user's profile and organisation. |
| **GET** | `/credits/balance` | JWT | Returns organisation credit balance and last 10 transactions. |
| **POST** | `/credits/grant` | **Admin** | Adds credits to the organisation. Body: { amount, reason }. |
| **POST** | `/api/analyse` | JWT | Synchronous product endpoint. Costs 25 credits. |
| **POST** | `/api/summarise` | JWT | Async product endpoint. Costs 10 credits. Returns job_id. |
| **GET** | `/api/jobs/{job_id}` | JWT | Polls job status. Returns result when completed. |

## Failure Modes — Read This Section

**This is not optional**

*Handling failure modes correctly carries the same weight as functional correctness in how your submission is evaluated.*

*A system that works perfectly when everything goes right but crashes or produces wrong results when things go wrong is not production-ready. Every scenario below must be handled explicitly.*

For every scenario listed below, your system must respond correctly. ‘Correctly’ means: the right HTTP status code, a structured error response, no data corruption, and the system continues operating normally afterwards.

### Authentication failures

- JWT token is missing from the request
- JWT token is present but expired
- JWT token is present, not expired, but the signature has been tampered with
- JWT token is valid but the user has been removed from the organisation (user_id no longer exists in the database)

### Credit system failures

- POST /api/analyse is called with exactly 0 credits remaining
- POST /api/analyse is called with 24 credits remaining (one less than required)
- Two simultaneous requests arrive for the same organisation, each requiring 25 credits, and the organisation has exactly 25 credits total — only one should succeed
- POST /api/summarise is called, the job is queued, but the background worker fails before processing completes

### Input validation failures

- POST /api/analyse is called with text shorter than 10 characters
- POST /api/analyse is called with text longer than 2000 characters
- POST /api/analyse is called with a missing text field entirely
- GET /api/jobs/{job_id} is called with a job_id that does not exist
- GET /api/jobs/{job_id} is called with a valid job_id that belongs to a different organisation

### Infrastructure failures

- The database is temporarily unreachable — what does /health return?
- Redis is temporarily unreachable — does the API keep working or go down entirely?
- A background job is still pending after 5 minutes — what is the job status and what happens to the credits?

## Written Decision Log

The decision log is a required part of your submission. It is not a README. It is not a description of what you built. It is a written record of the decisions you made, why you made them, and what you considered before deciding.

It should be between 600 and 1200 words. Shorter is acceptable if it is dense and specific. Longer is not acceptable.

Write it in plain English. No bullet-point lists. No code blocks. Paragraphs only. You are being evaluated on whether you can think clearly and communicate that thinking in writing — not on whether you can format a document.

## The five questions you must answer

**Question 1**

**Why does the credit system use a transaction ledger instead of a balance column?**

*Explain the design decision behind CreditTransactions. What problem does a ledger solve that a simple balance column does not? What are the tradeoffs?*

**Question 2**

**How did you handle the simultaneous credit deduction problem?**

*Two requests arrive at exactly the same time. The organisation has exactly 25 credits. Both requests cost 25 credits. Walk through exactly what happens in your implementation. What database mechanism prevents both from succeeding? Why did you choose that mechanism over alternatives?*

**Question 3**

**What happens when the background worker fails after credits have been deducted?**

*The job was queued. Credits were taken. The worker crashed before completing. Describe what your system does in this scenario. Did you implement a refund? A retry? Something else? Why?*

**Question 4**

**How does your idempotency implementation work, and where does it live?**

*Walk through the flow of a duplicate request with the same Idempotency-Key. Where is the check performed? What database constraint backs it up? What happens if two identical requests arrive simultaneously — before either has completed?*

**Question 5**

**What would break first at 10x the current load, and what would you do about it?**

*Assume your system is handling 10 times as many requests. You don't need to implement anything. Just identify the most likely bottleneck and describe in one paragraph what you would do about it.*

## How You Will Be Evaluated

Your submission is evaluated across four areas. The weighting is not published as a breakdown, but you should know that functional correctness alone is not enough to advance. How you handle failure and how clearly you can explain your decisions carry equal weight.

| Area | Weight | What we are looking at |
|---|---|---|
| **Functional correctness** | **High** | Do the endpoints work as specified? Does authentication function correctly? Does the credit system enforce the rules? |
| **Failure mode handling** | **High** | Are all failure scenarios handled explicitly? Do errors return the right status codes and structured responses? Does the system continue operating after a failure? |
| **Written reasoning** | **High** | Can you explain why you built it the way you did? Are your design decisions justified? Is your writing clear and specific? |
| **Code structure and hygiene** | **Moderate** | Is the codebase organised? Are secrets in environment variables? Is the README sufficient to run the project locally? |

**On partial completion**

*A submission that completes Tier 1 and Tier 2 well — with all failure modes handled and a clear decision log — will score higher than a submission that attempts Tier 3 but has broken authentication and no error handling.*

*Depth beats breadth. Get the fundamentals right before reaching for the hard requirement.*

## Submission Requirements

### What to submit

Your submission must contain three things. All three are required. A submission missing any one of them will not be evaluated.

1. A GitHub repository containing your complete codebase. The repository must be public or shared with the reviewer before the deadline.
2. A deployed, live URL where the API is running. It must respond to requests at the time of evaluation. Free-tier hosting on Railway, Render, or Fly.io is acceptable. Cloud Run is preferred but not required.
3. A written decision log answering the five questions above. This must be a plain text or Markdown file in the root of your repository named DECISIONS.md.

## Your README must include

- How to run the project locally (exact commands, not general instructions)
- What environment variables are required and what each one does
- How to run the database migrations
- At least three example API calls using curl or a similar tool

## Suggested timeline

This is a suggestion, not a requirement. Use it if you are unsure how to structure your time.

| Day | Suggested focus |
|---|---|
| **Day 1** | Read the entire assignment. Set up the project. Create the database schema and run the first Alembic migration. Get /health working. |
| **Day 1** | Google OAuth and JWT. /auth/google, /auth/callback, /me. Test the full login flow end to end before moving on. |
| **Day 2** | Credit system. CreditTransactions table, deduct_credits service function, /credits/balance, /credits/grant. Test the concurrent deduction case. |
| **Day 2** | Product endpoints. POST /api/analyse (synchronous). POST /api/summarise with ARQ background job. GET /api/jobs/{job_id}. |
| **Day 3** | Rate limiting. Structured logging. All failure modes from the failure modes section. Deploy to a live URL. |
| **Day 4** | Idempotency (Tier 3). Write the DECISIONS.md. Review the README. |
| **Day 5** | Buffer. Fix anything broken. Re-read the failure modes section and verify each one. Submit. |

| | |
|---|---|
| **Deadline and late** | *Submit before the deadline communicated with your application. Late submissions will not be evaluated.* |

| | |
|---|---|
| **submissions** | *If you encounter a genuine blocker (a third-party service outage, an account issue), email before the deadline — not after.* |

## What Happens After You Submit

Submissions are evaluated within one week of the deadline. You will receive a response regardless of the outcome.

| Stage | What happens |
|---|---|
| **Assignment** | You build and submit NexusAPI with a live URL, GitHub repo, and DECISIONS.md. |
| **Evaluation** | Submissions are scored across the four areas. The top 10 are selected for interview. |
| **Interview** | A technical interview based on your submission. You will be asked to walk through specific decisions from your DECISIONS.md and to explain parts of your code. There are no trick questions — only questions about what you built. |
| **Selection** | 2–3 candidates are offered internship positions on the backend engineering track. |

| | |
|---|---|
| **On the interview** | *The interview is not a separate coding test. It is a conversation about what you already submitted.*<br>*The evaluator will have read your DECISIONS.md before the interview. They will ask you to go deeper on specific answers. The candidates who advance from interview to offer are almost always the ones whose written reasoning was specific and honest — including about what they did not get right.* |

KASPARRO AI

# Backend Engineering

**“Build something you can explain.”**

rajat@kasparro.com